

Reviewer Pack — Minimal Ehrhart Polynomial Degree and Resolution Proof Width...

Entry d164440bf8cc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 04:13:40 UTC

Minimal Ehrhart Polynomial Degree and Resolution Proof Width Lower Bound

Entry ID: d164440bf8cc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 04:13:40 UTC

1. Conjecture

Field A (mathematical branch): Ehrhart Theory **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For every Boolean satisfiability problem with n variables, the resolution proof width of its corresponding Tseitin formula is lower-bounded by the minimal degree of an Ehrhart polynomial associated with any incidence matrix of the Tseitin construction.

Rationale (proposer's reasoning):

Ehrhart Theory provides a tool for counting integer points in algebraic sets. The association between Ehrhart polynomials and Boolean satisfiability through the Tseitin construction suggests a potential link between arithmetic properties of the problem and proof complexity, potentially revealing new insights into resolution proof mechanisms.

Taxonomy category: EHRHART_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4cbea2ee710d823b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$, the ratio of the minimum Ehrhart polynomial degree to the resolution proof width is greater than or equal to 1.0 for at least 90% of the generated Tseitin formulas.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Ehrhart Theory" AND "resolution proof complexity" - "Tseitin formula" AND "minimal Ehrhart polynomial degree" - "Boolean satisfiability problem" AND "incidence matrix" AND "Ehrhart polynomial"

Top relevant hits considered: - [s2:10.4230/LIPIcs.ITCS.2024.74] Intersection Classes in TFNP and Proof Complexity - [s2:2201.00913] Proof complexity of CSP

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i+1}' for i in range(n)]
        clauses = []

        # Generate clauses for each variable
        for i in range(n):
            clauses.append([variables[i]])
            clauses.append([-variables[i], variables[0], variables[i]])

        # Generate clauses to link all variables together
        for i in range(1, n):
            clauses.append([-variables[i-1], -variables[i], variables[i]])

        return variables, clauses

    def incidence_matrix(clauses):
        m = len(clauses)
        n = len(clauses[0])
        matrix = [[0] * n for _ in range(m)]

        for i, clause in enumerate(clauses):
            for var in clause:
                if var.startswith('x'):
                    j = int(var[1:]) - 1
                    matrix[i][j] = 1
```

```

        else:
            j = int(var[1:]) - 1
            matrix[i][j] = -1

    return matrix

def ehrhart_polynomial_degree(matrix):
    m, n = len(matrix), len(matrix[0])
    if m != n:
        raise ValueError("Matrix must be square")

    # Gaussian elimination to find the rank of the matrix
    for i in range(n):
        # Find a pivot
        pivot_row = next((j for j in range(i, n) if matrix[j][i] != 0), None)
        if pivot_row is None:
            return 0 # Matrix is singular

        # Swap rows to put the pivot at the diagonal
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]

        # Eliminate entries below the pivot
        for j in range(i + 1, n):
            factor = -matrix[j][i] / matrix[i][i]
            for k in range(n):
                matrix[j][k] += factor * matrix[i][k]

    return n - sum(1 for row in matrix if all(x == 0 for x in row))

def resolution_proof_width(clauses):
    # Simplified heuristic to estimate resolution proof width
    return len(max(set(sum(clause, [])), key=len))

variables, clauses = generate_tseitin_formula(5) # Start with n=5 for simplicity
matrix = incidence_matrix(clauses)
degree = ehrhart_polynomial_degree(matrix)
width = resolution_proof_width(clauses)

return {
    "metric_name": "ratio",
    "metric_value": degree / width if width != 0 else float('inf'),
    "instances_tested": 1,
    "n_max": 5,
    "conjecture_holds": degree >= width,
    "counterexample": "" if degree >= width else "resolution_width_exceeds_degree"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

```

```

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.9:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fea6dd80.py", line 99, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fea6dd80.py", line 79, in run_trial
    variables, clauses = generate_tseitin_formula(5) # Start with n=5 for simplicity
                          ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fea6dd80.py", line 28, in generate_tseitin_formula
    clauses.append([-variables[i], variables[0], variables[i]])
                   ^^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the error in the test code to ensure it can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16711
2	preregistration	ollama_remote	glm4:latest	0	0	9162
3	novelty	ollama_remote	glm4:latest	0	0	8325
4	novelty	ollama_remote	glm4:latest	0	0	9624
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30649
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13618
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14076
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11690
9	judge	ollama_remote	glm4:latest	0	0	13715

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 127569 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d164440bf8cc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d164440bf8cc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d164440bf8cc.tar.gz` (if generated)